

WEEK#06b**BASIC ARITHMETIC & LOGICAL OPERATIONS****I. Addition Operations using Assembly Instructions**

Half-Adder is implemented using the instruction ADD. No carry-in but carry-out may occur.
Full Adder is implemented using the instruction ADC. Both carry-in and carry-out may occur.

II. Subtraction Operations using Assembly Instructions

Half-Subtractor is implemented using the instruction SUB.

Full Subtractor is implemented using the instruction SBB.

Subtraction is implemented using 2's complement method.

Steps involved in subtraction using 2's complement are as under:

- Takes 2's complement of subtrahend
- Add it to the minuend.
- Invert the carry.
 - ✓ If CY=0 then result is a positive number
 - ✓ Else if CY=1 then result is a negative number and stored in 2's complement.

There could be two cases as described in Table 7.1.

Table 7.1 Two cases of unsigned subtraction

CASE 1	CASE 2
32 -14 = 32 + (-14)	-32 +14
00100000 (32)	11100000 (-32)
<u>11110010</u> (-14)	<u>00001110</u> (+14)
<u>00010010</u> (+18)	<u>11101110</u> (-18)
CY = 1	CY = 0
inverting CY = 0	inverting CY = 1
Answer is +ve	Answer is -ve
00010010	11101110 (2's complement)

III. Multiplication Operations using Assembly Instructions

MUL instructions is used for multiplication of unsigned numbers with the syntax

MUL <operand 2>. Operand 1 must be held in either AL or AX as depicted in Table 7.2.

Table 7.2 Operands used for MUL instruction

Multiplication	Operand 1	Operand 2	Result
Byte x byte	AL	Register or memory	AX
Word x word	AX	Register or memory	DX AX
Word x byte	AL=byte, AH=0	Register or memory	DX AX

IV. Division Operations using Assembly Instructions

DIV instructions is used for multiplication of unsigned numbers with the syntax

DIV <Denominator>. Numerator must be held in either AL, AX or combination of AX and DX as depicted in Table 7.3.

Table 7.3 Operands used for DIV instruction

Division	Numerator	Denominator	Quotient	Remainder
Byte / byte	AL=byte, AH=0	Register or memory	AL	AH
Word / word	AX=word,DX=0	Register or memory	AX	DX
Word / byte	AX=word	Register or memory	AL	AH
DW / word	DX:AX=DW	Register or memory	AX	DX

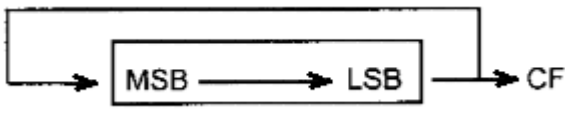
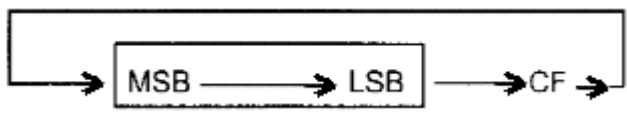
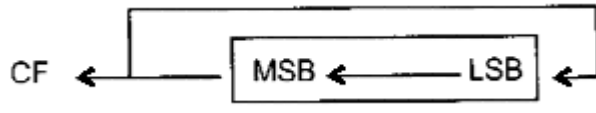
NOTE: DW stands for Double Word

V. Logical Instructions in Assembly Language

Assembly language implements Logical operations using the following instructions:

Logical Instructions	Description															
NOT	Provides 1's complement of given operand NOT destination															
AND	Provides 1 if both operand bits are 1 else provides 0 as output AND destination,source <table><tr><td>X</td><td>Y</td><td>X AND Y</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	X AND Y	0	0	0	1	0	0	0	1	0	1	1	1
X	Y	X AND Y														
0	0	0														
1	0	0														
0	1	0														
1	1	1														
OR	Provides 1 if any of operand bits is 1 else provides 0 as output OR destination,source <table><tr><td>X</td><td>Y</td><td>X OR Y</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	X OR Y	0	0	0	1	0	1	0	1	1	1	1	1
X	Y	X OR Y														
0	0	0														
1	0	1														
0	1	1														
1	1	1														

Logical Instructions	Description															
XOR	Provides 1 if operand bits are not equal else provides 0 as output XOR destination,source <table><tr><td>X</td><td>Y</td><td>X XOR Y</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	X XOR Y	0	0	0	1	0	1	0	1	1	1	1	0
X	Y	X XOR Y														
0	0	0														
1	0	1														
0	1	1														
1	1	0														
CMP	Compare two operands by subtracting the operands and analyzing the carry and zero flags. CMP destination, source <table><tr><td>Operands</td><td>CF (CARRY FLAG)</td><td>ZF (ZERO FLAG)</td></tr><tr><td>Destination > source</td><td>0</td><td>0</td></tr><tr><td>Destination = source</td><td>0</td><td>1</td></tr><tr><td>Destination < source</td><td>1</td><td>0</td></tr></table>	Operands	CF (CARRY FLAG)	ZF (ZERO FLAG)	Destination > source	0	0	Destination = source	0	1	Destination < source	1	0			
Operands	CF (CARRY FLAG)	ZF (ZERO FLAG)														
Destination > source	0	0														
Destination = source	0	1														
Destination < source	1	0														
SHR	Shift bit of the operand (<i>register or memory location</i>) rightward once, LSB goes to Carry Flag (CF) and MSB is filled with zero SHR AL, 1 CL can used to hold the count if the bits are required to be shifted more than once MOV CL, 4 SHR AL, CL 0 → MSB → LSB → CF															
SHL	Shift bit of the operand (<i>register or memory location</i>) leftward once, MSB goes to Carry Flag (CF) and LSB is filled with zero SHL AL, 1 CL can used to hold the count if the bits are required to be shifted more than once MOV CL, 4 SHL AL, CL CF ← MSB ← LSB ← 0															

Logical Instructions	Description
ROR	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) ROR AL, 1 
RCR	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) RCR AL, 1 
ROL	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) ROL AL, 1 
RCL	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) RCL AL, 1 